

claude-windows / 96963dab-d38f-488c-af44-530ddbefe1

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbefe1\subagents\workflows\wf_dd531439-5e0\agent-a1ef770fff54d47e0.jsonl`
- SHA-256: `57f8164e168f41f7172b1d5fde8488b7719dd9913495db44c1a45ef5439b88df`
- Source modified: `2026-07-08T06:41:05+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_dd531439-5e0`
- session_id: `96963dab-d38f-488c-af44-530ddbefe1`
```

Transcript

1. user (2026-07-08T06:36:07.019Z)

You are reviewing an UNCOMMITTED diff in an isolated git worktree at C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on origin/master).

To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and `git --no-pager diff --stat`; read the new files docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly. Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata md5 converted to hex by backend/storage/gcs.py::_md5_hex) instead of re-hashing the fetched clip, gated by deployment.trust_submit_video_id (default True) and guarded by orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-l4 (phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design. For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say so explicitly.

A prior reviewer raised this finding on the REPO CONVENTIONS / DISCIPLINE dimension:

SEVERITY: nit

FILE: backend/config/models.py:493

SUMMARY: trust_submit_video_id is appended at the tail of DeploymentConfig, the same insertion point #521 uses to extend DeploymentConfig ? a likely (trivial) textual merge conflict.

CLAIMED FAILURE: The field is added at line 493, immediately before `class BillingConfig` (line 496). #521 also extends DeploymentConfig (phase_placement). Two additive tail-appends at the same class boundary produce a textual git conflict when the second of #521/#524 merges, even though the additions are semantically independent.

PROPOSED FIX: No code change needed ? flag for whoever merges second: keep both field additions (distinct names, both pure additions). Optionally rebase-order so one lands first and the other re-appends cleanly.

Adversarially VERIFY it against the real code/doc in

C:/Users/avidu/AppData/Local/Temp/claude/wt524-969. Default to REFUTED unless you can construct the concrete failing input/state and confirm it in the actual code. A finding is CONFIRMED only if it is a real defect a maintainer must fix (not a style preference, not already-handled, not a misread). State the verdict and the evidence (file:line you checked).

2. user (2026-07-08T06:36:13.194Z)

440 # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.

```

441 workspace_root: str = ""
442 # Cloud namespace prefix under the root so per-video folders sit tidily beside
443 # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
444 workspace_prefix: str = "workspaces"
445 local: dict[str, Any] = Field(default_factory=dict)
446 s3: dict[str, Any] = Field(default_factory=dict)
447 gcs: dict[str, Any] = Field(default_factory=dict)
448 gdrive: dict[str, Any] = Field(default_factory=dict)
449 # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
450 # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
451 gdrive_api: dict[str, Any] = Field(default_factory=dict)
452
453
454 class DeploymentConfig(BaseModel):
455     """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456     knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
457
458     **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
459     behaves exactly as before this section existed. A profile is opt-in (``config.json``
460     ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
461     *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
462     these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
463     (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
464
465     Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
466
467     # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469     # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470     # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472     # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473     # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474     # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475     # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476     # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477     max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478     # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
479     # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
480     # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
483     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
484     # Trust the SUBMIT-TIME md5 instead of re-hashing the fetched file on the drain
(#524 win #2).

```

```

485         # The chunked upload already streams the whole-file md5
(``backend/api/uploads.py``), and the
486         # submit hook records it on the job row (``submit_time_video_id``, one gs://
metadata stat ? #484);
487         # that value is BYTE-IDENTICAL to ``compute_video_id`` (same bytes, same MD5) and is
ALREADY
488         # trusted for the submit-time cache probe (``backend/api/jobs.py``). When True
(default) the drain
489         # reuses it and SKIPS re-hashing the multi-GB clip it just fetched to scratch; when
the id is absent
490         # (a local/appliance upload or a non-gcs direct-URI submit ?
``submit_time_video_id`` returns None)
491         # the drain still hashes, exactly as before. Set False to force a re-hash
unconditionally
492         # (belt-and-suspenders rollback ? reverts to pre-#524 behaviour with no redeploy).
493         trust_submit_video_id: bool = True
494
495
496     class BillingConfig(BaseModel):
497         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
498
499         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
500         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
501         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
502         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
503         inserts a real, calibrated version and points ``pricebook_version`` at it.
504
505         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
506         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
507         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
508
509         enabled: bool = False

```

3. user (2026-07-08T06:36:22.999Z)

warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test_video_id_trust.py', LF will be replaced by CRLF the next time Git touches it

```

backend/api/job_worker.py          | 5 +
backend/config/models.py           | 10 +
backend/orchestration.py           | 51 ++++
.../L4_PRIMARY_TOPOLOGY.md        | 225 +++++
docs/COMPUTE_DECOUPLED_SERVING/README.md | 2 +
tests/test_config_deployment.py    | 2 +
tests/test_video_id_trust.py       | 173 +++++
7 files changed, 466 insertions(+), 2 deletions(-)

```

====DIFF models.py====

diff --git a/backend/config/models.py b/backend/config/models.py

index a7c997c..71e43b8 100644

--- a/backend/config/models.py

+++ b/backend/config/models.py

@@ -481,6 +481,16 @@ class DeploymentConfig(BaseModel):

1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect + queue/cold-start,

observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid GPU run.
remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)

+ # Trust the SUBMIT-TIME md5 instead of re-hashing the fetched file on the drain (#524 win

```

#2).
+ # The chunked upload already streams the whole-file md5 (`backend/api/uploads.py`), and
the
+ # submit hook records it on the job row (`submit_time_video_id`, one gs:// metadata stat
? #484);
+ # that value is BYTE-IDENTICAL to `compute_video_id` (same bytes, same MD5) and is
ALREADY
+ # trusted for the submit-time cache probe (`backend/api/jobs.py`). When True (default)
the drain
+ # reuses it and SKIPS re-hashing the multi-GB clip it just fetched to scratch; when the id
is absent
+ # (a local/appliance upload or a non-gcs direct-URI submit ? `submit_time_video_id`
returns None)
+ # the drain still hashes, exactly as before. Set False to force a re-hash unconditionally
+ # (belt-and-suspenders rollback ? reverts to pre-#524 behaviour with no redeploy).
+ trust_submit_video_id: bool = True

```

```
class BillingConfig(BaseModel):
```

4. user (2026-07-08T06:36:36.266Z)

```

chore/revendor-hosted-compute-picker
+ claude/compile-reliability
+ claude/wasb-config-validation
+ codex/issue-523-sd-reel-plan
+ feat/521-compile-on-l4
* feat/524-l4-primary
  feat/submit-md5-cache-reuse
+ fix/521-worker-mirror-required
  fix/cloud-run-exec-status-fastfail
  fix/hosted-local-detection-guardrail
+ fix/nvenc-probe-resolution
  master
+ perf/521-cheap-watermark
  perf/validation-shared-decode-pass
  remotes/origin/HEAD -> origin/master
  remotes/origin/chore/revendor-hosted-compute-picker
  remotes/origin/docs/runlog-cloud-run-exec-status-fastfail
  remotes/origin/docs/validation-latency-live-cuj-2026-07-08
  remotes/origin/feat/521-compile-on-l4
  remotes/origin/feat/validation-result-cache
  remotes/origin/fix/cloud-run-exec-status-fastfail
  remotes/origin/fix/hosted-local-detection-guardrail
  remotes/origin/fix/nvenc-probe-resolution
  remotes/origin/master
=====remotes=====
  origin/HEAD -> origin/master
  origin/chore/revendor-hosted-compute-picker
  origin/docs/runlog-cloud-run-exec-status-fastfail
  origin/docs/validation-latency-live-cuj-2026-07-08
  origin/feat/521-compile-on-l4
  origin/feat/validation-result-cache
  origin/fix/cloud-run-exec-status-fastfail
  origin/fix/hosted-local-detection-guardrail
  origin/fix/nvenc-probe-resolution
  origin/master

```

5. user (2026-07-08T06:36:41.464Z)

```

=====521 diff on models.py vs master=====
diff --git a/backend/config/models.py b/backend/config/models.py
index a7c997c..ce8e98a 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py

```

```

@@ -364,6 +364,14 @@ class StitchingConfig(BaseModel):
    # background-court fragments), so this keeps the reel to the eye-catching long rallies.
    # OFF by default ? the detector output is unchanged, only which rallies reach the reel.
    min_rally_duration: float = Field(default=0.0, ge=0.0)
+   # Video encoder for the per-clip trim re-encode (the branding watermark forces a re-encode;
the
+   # concat stays -c copy). Default ``libx264`` (CPU, works everywhere incl. CI).
``h264_nvenc`` /
+   # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-encode
that
+   # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested encoder
ONCE and
+   # falls back to libx264 if this ffmpeg build can't run it, so a non-GPU box never breaks.
The
+   # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for the L4
run;
+   # the control-plane's own local stitch keeps libx264.
+   encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
+   watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)

```

```

@@ -451,6 +459,21 @@ class StorageConfig(BaseModel):
    gdrive_api: dict[str, Any] = Field(default_factory=dict)

```

```

+class PhasePlacementConfig(BaseModel):
+   """Per-phase machine placement (#521): move validation / segment(detect) / compile(stitch)
+   between the control-plane and the L4 Cloud Run worker **by config, no code change**.
+
+   Each field is ``None`` (? the legacy default for that phase ? see
+   ``backend/config/placement.py``) or an explicit ``"control_plane"`` / ``"cloud_run_l4"``.
+   ``control_plane`` runs the phase in-process on the CPU-only control-plane; ``cloud_run_l4``
+   dispatches it to the GPU/NVENC worker Job. Unset ? byte-identical to pre-#521 behaviour.
+   Resolve via ``backend.config.resolve_phase_placement(cfg, phase)`` ? never read these
directly."""
+
+   validation: Optional[Literal["control_plane", "cloud_run_l4"]] = None
+   segment: Optional[Literal["control_plane", "cloud_run_l4"]] = None
+   compile: Optional[Literal["control_plane", "cloud_run_l4"]] = None
+
+
+   class DeploymentConfig(BaseModel):
+       """Deployment profile ? one switch that selects a coherent bundle of compute + storage
+       knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).

```

```

@@ -481,6 +504,35 @@ class DeploymentConfig(BaseModel):
    # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect + queue/cold-
start,
    # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid GPU run.
    remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
+   # Per-phase machine placement (#521). Unset ? today's placement (validation + compile on
the
+   # control-plane, segment on the L4 iff compute_target==cloud_run). Set a phase to relocate
it by
+   # config alone. Resolve via ``backend.config.resolve_phase_placement`` (never read the sub-
fields
+   # directly ? the resolver folds in the legacy compute_target default).
+   phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
+   # Video encoder the cloud compile-dispatch forwards onto ``stitching.encoder`` for the L4
stitch
+   # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the 4K re-encode the watermark
forces;
+   # set ``libx264`` to run the L4 stitch on CPU. Only consulted when compile is placed on the
L4;
+   # the control-plane's own local stitch always uses ``stitching.encoder`` (libx264 by
default).

```

```

+   cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
+
+   @model_validator(mode="after")
+   def _validation_placement_needs_cloud_segment(self) -> "DeploymentConfig":
+       # A clip can only be validated ON the worker if the worker also DETECTS it there ? the
worker
+       # is only launched for the segment(detect) phase. So validation=cloud_run_l4 with
segment on
+       # the control-plane would leave NOBODY validating (the control-plane skips, and no
worker runs)
+       # ? reject that footgun at config load rather than ship a silently-unvalidated clip.
+       if self.phase_placement.validation == "cloud_run_l4":
+           seg = self.phase_placement.segment or (
+               "cloud_run_l4" if self.compute_target == "cloud_run" else "control_plane"
+           )
+       if seg != "cloud_run_l4":
+           raise ValueError(
+               "deployment.phase_placement.validation='cloud_run_l4' requires segment to
also "
+               "run on the L4 ? the worker can only validate a clip it also detects. Set "
+               "phase_placement.segment='cloud_run_l4' (or compute_target='cloud_run'), or
keep "
+               "validation on the control_plane."
+           )
+       return self

```

```
class BillingConfig(BaseModel):
```

6. assistant (2026-07-08T06:36:54.162Z)

Let me verify the actual insertion points and empirically test whether a merge conflict occurs.

7. user (2026-07-08T06:36:57.776Z)

```

=====HEAD / status=====
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
($9) (#522)
M backend/api/job_worker.py
M backend/config/models.py
M backend/orchestration.py
A docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md
M docs/COMPUTE_DECOUPLED_SERVING/README.md
M tests/test_config_deployment.py
A tests/test_video_id_trust.py
=====merge-base check=====
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)

```

8. assistant (2026-07-08T06:37:07.132Z)

Origin/master has advanced ? #521 already merged as #525 (`dac6736`). Let me verify the relationship and empirically test the merge/rebase.

9. user (2026-07-08T06:37:11.128Z)

```

=====is bf97700 ancestor of origin/master?=====
YES bf97700 is ancestor of origin/master
=====commits on origin/master since worktree HEAD=====
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
=====does origin/master already have phase_placement in models.py?=====
origin/master:backend/config/models.py:470: Resolve via
`backend.config.resolve_phase_placement(cfg, phase)` ? never read these directly.

```

```

origin/master:backend/config/models.py:506:     remote_poll_max_wait_sec: float =
Field(default=3900.0, gt=0)
origin/master:backend/config/models.py:509:     # config alone. Resolve via
``backend.config.resolve_phase_placement`` (never read the sub-fields
origin/master:backend/config/models.py:511:     phase_placement: PhasePlacementConfig =
Field(default_factory=PhasePlacementConfig)
origin/master:backend/config/models.py:524:         if self.phase_placement.validation ==
"cloud_run_l4":
origin/master:backend/config/models.py:525:             seg = self.phase_placement.segment or (
origin/master:backend/config/models.py:530:
"deployment.phase_placement.validation='cloud_run_l4' requires segment to also "
origin/master:backend/config/models.py:532:
"phase_placement.segment='cloud_run_l4' (or compute_target='cloud_run'), or keep "

```

10. assistant (2026-07-08T06:37:28.507Z)

#521 already landed on origin/master (as #525), so #524 is now the second-to-merge ? exactly the scenario the finding predicts. This is empirically testable. Let me run the real 3-way merge on models.py.

11. user (2026-07-08T06:37:36.115Z)

```

===== running 3-way merge: OURS=master(521) BASE=bf97700 THEIRS=524working =====
EXIT=1
===== conflict markers? =====
1:<<<<<< C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer/96963dab-d38f-488c-af44-530ddbeefed1/scratchpad/master521.py
615:=====
1187:>>>>>> C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer/96963dab-d38f-488c-af44-530ddbeefed1/scratchpad/mine524.py

```

12. assistant (2026-07-08T06:37:42.367Z)

The conflict spans nearly the whole file (lines 1-1187), which smells like a CRLF/LF artifact from the working-tree copy. Let me check line endings and redo with normalization.

13. user (2026-07-08T06:37:47.254Z)

```

===== CRLF check (count CR bytes) =====
base.py: CR=0
master521.py: CR=0
mine524.py: CR=0
EXIT=1
===== conflict markers after normalization =====
grep: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/96963dab-d38f-488c-af44-530ddbeefed1/scratchpad/n_mergeout.py: No such file or
directory

```

14. user (2026-07-08T06:38:09.863Z)

```

ls: cannot access 'n_*.py': No such file or directory
-rw-r--r-- 1 avidu 197609 33120 Jul  8 12:07 base.py
-rw-r--r-- 1 avidu 197609 36917 Jul  8 12:07 master521.py
-rw-r--r-- 1 avidu 197609 34627 Jul  8 12:07 mine524.py
===== line counts =====
 561 base.py
 613 master521.py
 571 mine524.py
1745 total
===== diff base vs master521 (first lines) =====
366a367,374
> # Video encoder for the per-clip trim re-encode (the branding watermark forces a re-
encode; the
> # concat stays -c copy). Default ``libx264`` (CPU, works everywhere incl. CI).

```

```

`h264_nvenc` /
> # `hevc_nvenc` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-encode
that
> # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested encoder
ONCE and
> # falls back to libx264 if this ffmpeg build can't run it, so a non-GPU box never breaks.
The
> # cloud compile-dispatch forwards `deployment.cloud_stitch_encoder` onto this for the L4
run;
> # the control-plane's own local stitch keeps libx264.
> encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
453a462,476
> class PhasePlacementConfig(BaseModel):
>     """Per-phase machine placement (#521): move validation / segment(detect) / compile(stitch)
>     between the control-plane and the L4 Cloud Run worker **by config, no code change**.
>
>     Each field is `None` (? the legacy default for that phase ? see
>     `backend/config/placement.py`) or an explicit `"control_plane"` / `"cloud_run_l4"`.
>     `control_plane` runs the phase in-process on the CPU-only control-plane;
`cloud_run_l4`
>     dispatches it to the GPU/NVENC worker Job. Unset ? byte-identical to pre-#521 behaviour.
>     Resolve via `backend.config.resolve_phase_placement(cfg, phase)` ? never read these
directly."""
>
>     validation: Optional[Literal["control_plane", "cloud_run_l4"]] = None
>     segment: Optional[Literal["control_plane", "cloud_run_l4"]] = None
>     compile: Optional[Literal["control_plane", "cloud_run_l4"]] = None
>
>
483a507,535
> # Per-phase machine placement (#521). Unset ? today's placement (validation + compile on
the
> # control-plane, segment on the L4 iff compute_target==cloud_run). Set a phase to relocate
it by
> # config alone. Resolve via `backend.config.resolve_phase_placement` (never read the
sub-fields
> # directly ? the resolver folds in the legacy compute_target default).
> phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
> # Video encoder the cloud compile-dispatch forwards onto `stitching.encoder` for the L4
stitch
> # (#521). `h264_nvenc` uses the L4's hardware encoder for the 4K re-encode the watermark
forces;
> # set `libx264` to run the L4 stitch on CPU. Only consulted when compile is placed on
the L4;
> # the control-plane's own local stitch always uses `stitching.encoder` (libx264 by
default).
> cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
>
> @model_validator(mode="after")
> def _validation_placement_needs_cloud_segment(self) -> "DeploymentConfig":
>     # A clip can only be validated ON the worker if the worker also DETECTS it there ? the
worker

```

15. assistant (2026-07-08T06:38:25.843Z)

The base-vs-521 diff is clean and localized. The `n\_` files failed to create (the whole-file conflict in run 1 is an artifact I need to pin down). Let me re-run merge-file cleanly on the confirmed-LF files and inspect the actual conflict region.

16. user (2026-07-08T06:38:31.578Z)

```

EXIT=1
===== marker lines =====
1:<<<<<< master521.py
615:=====

```



```
1187:>>>>>> mine524.py
===== total merged lines =====
1187 mergeout2.py
```

17. assistant (2026-07-08T06:38:51.190Z)

A whole-file conflict despite clean localized diffs means `git merge-file` is misbehaving on these files. Let me run a sanity check to understand the tool, then use git's real merge machinery instead.

18. user (2026-07-08T06:38:56.071Z)

```
===== SANITY: merge base/base/master521 (OURS==BASE, should be clean) =====
EXIT=0 (0=clean)
===== first bytes hexdump of each =====
base.py: 00000000: 2222 2250 7964 616e 7469 6320 6d6f 6465  """Pydantic mode
master521.py: 00000000: 2222 2250 7964 616e 7469 6320 6d6f 6465  """Pydantic mode
mine524.py: 00000000: 2222 2250 7964 616e 7469 6320 6d6f 6465  """Pydantic mode
===== last bytes (final newline?) =====
base.py: 00000000: 7043 6f6e 6669 670a                                pConfig.
master521.py: 00000000: 7043 6f6e 6669 670a                                pConfig.
mine524.py: 00000000: 436f 6e66 6967 0d0a                                Config..
```

19. assistant (2026-07-08T06:39:10.045Z)

The working-tree copy `mine524.py` ends in CRLF while the git blobs are LF ? that's the whole-file-conflict artifact (Windows checkout). The committed #524 blob will be LF (matching base). Let me normalize to LF and run the true merge.

20. user (2026-07-08T06:39:16.387Z)

```
last bytes mine524_lf:
00000000: 7043 6f6e 6669 670a                                pConfig.
===== TRUE 3-way merge: OURS=master521 BASE=base THEIRS=mine524_lf =====
EXIT=1 (0=clean,1=conflict)
===== conflict markers =====
507:<<<<<< master521.py
537:=====
548:>>>>>> mine524_lf.py
===== context around DeploymentConfig tail in merge output =====
372:  # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for the
L4 run;
470:  Resolve via ``backend.config.resolve_phase_placement(cfg, phase)`` ? never read these
directly."""
506:  remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
507:<<<<<< master521.py
510:  # config alone. Resolve via ``backend.config.resolve_phase_placement`` (never read the
sub-fields
512:  phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
517:  cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
525:      if self.phase_placement.validation == "cloud_run_l4":
526:          seg = self.phase_placement.segment or (
531:              "deployment.phase_placement.validation='cloud_run_l4' requires segment
to also "
533:              "phase_placement.segment='cloud_run_l4' (or compute_target='cloud_run'),
or keep "
537:=====
547:  trust_submit_video_id: bool = True
548:>>>>>> mine524_lf.py
551:class BillingConfig(BaseModel):
```

21. assistant (2026-07-08T06:40:12.859Z)

The true LF merge produces a **real, localized conflict** at the `DeploymentConfig` tail

(models.py, merged lines 507?548): #521's `phase\_placement`/`cloud\_stitch\_encoder`/validator and #524's `trust\_submit\_video\_id` are both inserted immediately after the same anchor line `remote\_poll\_max\_wait\_sec` and before the same `class BillingConfig`. Let me confirm the scope ? whether #521 (now in master) also collides with the *other* files #524 touches.

22. user (2026-07-08T06:40:17.620Z)

```
===== files #524 modifies vs files #521(=master since bf97700) modifies =====
--- #524 (working tree) ---
M backend/api/job_worker.py
M backend/config/models.py
M backend/orchestration.py
A docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md
M docs/COMPUTE_DECOUPLED_SERVING/README.md
M tests/test_config_deployment.py
A tests/test_video_id_trust.py
--- #521 landed in origin/master since bf97700 ---
backend/compute/__init__.py
backend/compute/base.py
backend/compute/cloud_run.py
backend/config/__init__.py
backend/config/models.py
backend/config/placement.py
backend/config/presets.py
backend/orchestration.py
backend/pipeline/stitcher/compiler.py
backend/worker/__main__.py
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
docs/COMPUTE_DECOUPLED_SERVING/README.md
tests/test_api_cloud_dispatch.py
tests/test_compile_dispatch.py
tests/test_compiler.py
tests/test_compute_backend.py
tests/test_config_deployment.py
tests/test_phase_placement.py
tests/test_worker.py
===== overlap? both touch: =====
backend/config/models.py
backend/orchestration.py
docs/COMPUTE_DECOUPLED_SERVING/README.md
tests/test_config_deployment.py
```

23. assistant (2026-07-08T06:40:55.083Z)

I have everything I need. Let me record the verification.

****What I verified (empirically, in real code):****

- Worktree HEAD is `bf97700`; ****`origin/master` has already advanced to `dac6736` = "#521 ? config-driven phase?machine placement (#521) (#525)"****. So #521 is **already merged**; #524 is now unavoidably the second-to-merge ? the finding's premise is no longer hypothetical.
- ``origin/master:backend/config/models.py`` inserts #521's ``phase_placement``, ``cloud_stitch_encoder``, and a ``@model_validator`` immediately after ``remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)`` (blob line 506), before ``class BillingConfig``.
- #524 (working tree, models.py:493) inserts ``trust_submit_video_id: bool = True`` after that **same** anchor line, before that **same** ``class BillingConfig``.
- I ran the true 3-way merge (``git merge-file``, after normalizing the Windows-checkout CRLF on the working copy to LF to match the committed blob ? the first run's whole-file "conflict" was purely that CRLF/LF artifact). Result: ****exit 1, one localized conflict**** at merged lines 507?548, exactly at the ``DeploymentConfig`` tail ? ``<<<<<<< master521`` (phase_placement + cloud_stitch_encoder + validator) ``=====`` ``trust_submit_video_id`` ``>>>>>> mine524``. So the textual conflict is confirmed and now guaranteed.
- Scope: the rebase also collides in ``backend/orchestration.py``, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``, and ``tests/test_config_deployment.py`` (all touched by both), but the finding is scoped to models.py.

****Why this does not clear the "must-fix defect" bar:**** It is not a code defect. The #524 code is correct; appending a Pydantic field at a config class's tail is idiomatic, and the "blame" for the adjacency is symmetric between #521/#524. The two additions are independent, distinctly-named, pure-additive fields (#521's validator never references `trust\_submit\_video\_id`), so resolution is mechanically "keep both" with zero code change and zero risk ? exactly what the finding itself states ("No code change needed"). That is a merge-hygiene/process note, which the task excludes as style/discipline, not a real defect a maintainer must fix in the PR. The author must rebase onto the advanced `origin/master` regardless (three other files also conflict), and this hunk resolves by keeping both fields.

24. user (2026-07-08T06:41:05.269Z)

Structured output provided successfully